Cloud Task Placement

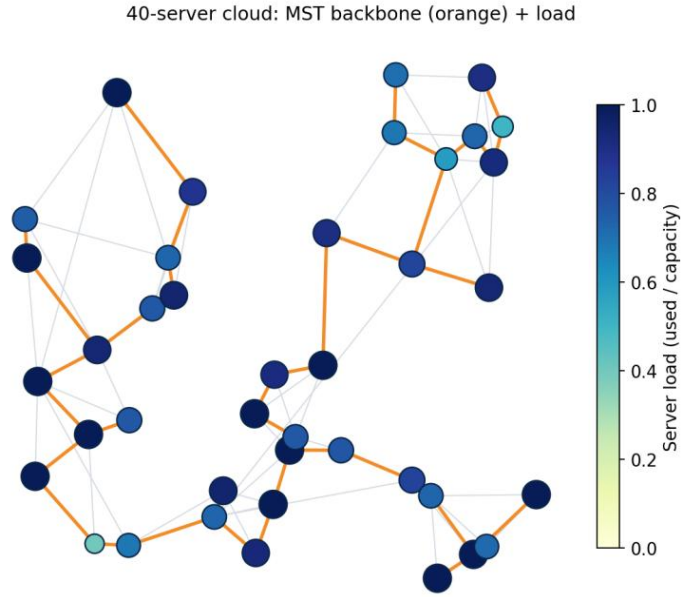
Latency-aware VM placement using graphs, greedy algorithms and dynamic programming

Design and Analysis of Algorithms | Internal Project

Team

1. Andrew Devasani | 25WU0102067
2. Sathyanarayana Varma | 25WU0102088
3. G. Daiwik Reddy | 25WU0102340
4. G. Keerthan Chowdary | 25WU0102076

B.Tech CSE (2025-2029), Woxsen University



40 servers: MST backbone in orange, darker = fuller

Problem statement

The problem

A cloud has servers connected by links with latencies. Each task needs some capacity, earns a value, and has users at a particular node. Decide which tasks to run and where, so total value minus latency cost is maximised and no server is overloaded.

Why it matters

Poor placement means slow responses and wasted servers. Exact solutions do not scale (the problem is NP-hard), so good heuristics matter.

1

Match algorithms to subproblems

MST for the backbone, shortest paths for latency, greedy and DP for placement

2

Respect real constraints

Server capacities and a latency penalty shape every placement

3

Measure the trade-offs

Time bounds, measured runtime and quality against the exact optimum

Background: algorithms used

Source	Contribution	Used for
Kruskal (1956)	Greedy MST by sorted edges + union-find	Backbone network
Prim (1957)	Grow an MST from one node	Cross-check of Kruskal
Dijkstra (1959)	Shortest paths, non-negative weights	Server-to-server latency
Floyd (1962), Warshall (1962)	All-pairs shortest paths by DP, O(V^3)	Dense-network latency matrix
Johnson (1977)	APSP for sparse graphs via reweighting	Sparse-network latency matrix
Martello and Toth (1990)	Knapsack problems: DP and bounds	Per-server task selection
Cormen et al., CLRS	Textbook proofs and complexity	Correctness arguments

Gap addressed: the textbook algorithms are usually taught separately; this project combines them in one placement pipeline and measures the trade-offs.

Software requirements (SRS)

Functional requirements

- FR1 Generate or load a server network and workload
- FR2 Build an MST backbone (Kruskal, Prim)
- FR3 Compute latency matrix (Dijkstra, Floyd-Warshall, Johnson)
- FR4 Place tasks within capacity (Greedy + DP)
- FR5 Compare with baselines and the exact optimum

Non-functional requirements

- Correct: tests cross-check every algorithm
- Fast: placement of 600 tasks on 80 servers in well under a second
- Reproducible: seeded synthetic data
- Modular: one file per algorithm family
- Python only, matplotlib for charts

Input: Topology (nodes, link latencies), server capacities, task demand, value and origin

Output: Task-to-server plan, utility, average latency, charts and policy tables

Dataset and model

Network (synthetic)

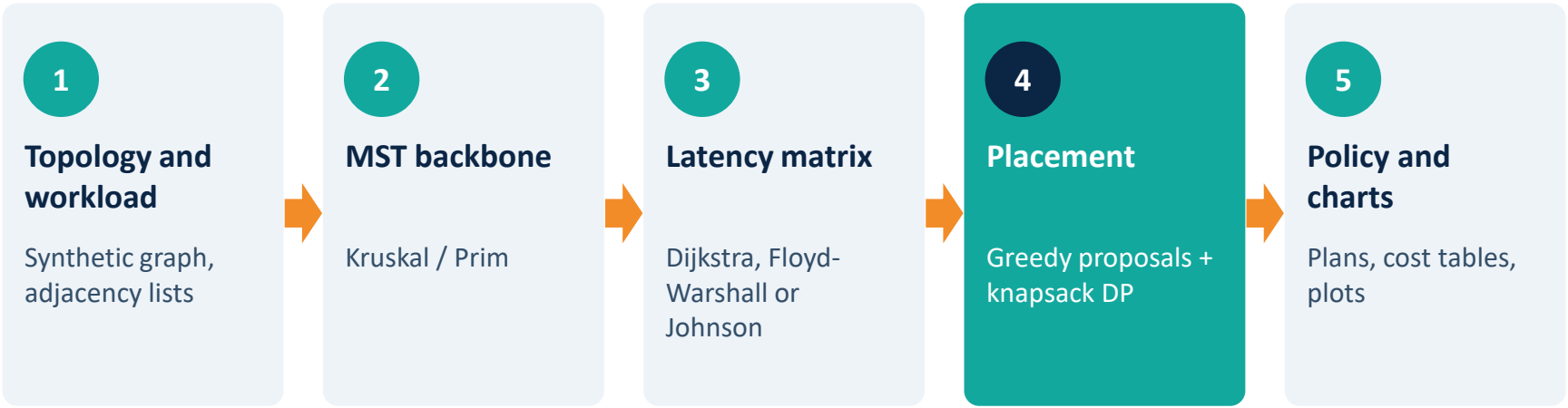
- Servers get random positions on a plane
- Latency = 1 ms + distance x 40 ms + 0-2 ms jitter
- Each server links to its nearest earlier server, so the network is connected
- Extra nearest-neighbour links set the density

Workload

- Task demand: 1 to 10 capacity units
- Task value: 5 to 60
- Users of each task sit at a random server
- Server capacity varies 0.6x to 1.4x of the average
- Load = total demand / total capacity

```
utility(t, s) = value(t) - lambda * latency(origin(t), s)
```

System design



Utility of placing task t on server s

```
utility(t, s) = value(t) - lambda * latency(origin(t), s)
```

lambda = 1; a task is placed only if its utility is positive

Backbone design with an MST

Kruskal

- Sort all links by latency
- Add a link if it joins two separate groups (union-find)
- Time O(E log E)

Prim

- Grow the tree from one server
- Always add the cheapest link leaving the tree (binary heap)
- Time O(E log V)

74%

fewer links than the full network, with only V-1 links kept

Use: a cheap backbone for replication and control traffic. It is not used for user latency, because routes on a tree are longer (see policy).

Algorithm choices and complexity

Subproblem	Algorithm	Time	Why
Backbone	Kruskal / Prim	O(E log E) / O(E log V)	Two methods agree: built-in check
Latency, one source	Dijkstra	O((V+E) log V)	Latencies are non-negative
Credits / penalties	Bellman-Ford	O(VE)	Negative edges, detects cycles
All pairs, dense	Floyd-Warshall	O(V^3)	Simple, wins near complete graphs
All pairs, sparse	Johnson	O(VE + V^2 log V)	Reweighting, then Dijkstra
Value upper bound	Fractional knapsack	O(n log n)	Greedy is optimal when divisible
Per-server selection	0-1 knapsack DP	O(nC)	Exact, items are indivisible
Deadline batch jobs	Job sequencing	O(n^2)	Greedy by profit, exchange argument

Placement algorithm and correctness

Proposal rounds

1. Each pending task proposes to its best server (highest positive utility, not banned)
2. Each server runs 0-1 knapsack DP over its proposals, with its remaining capacity
3. Chosen tasks are placed; the rest ban that server
4. Repeat until nothing is pending

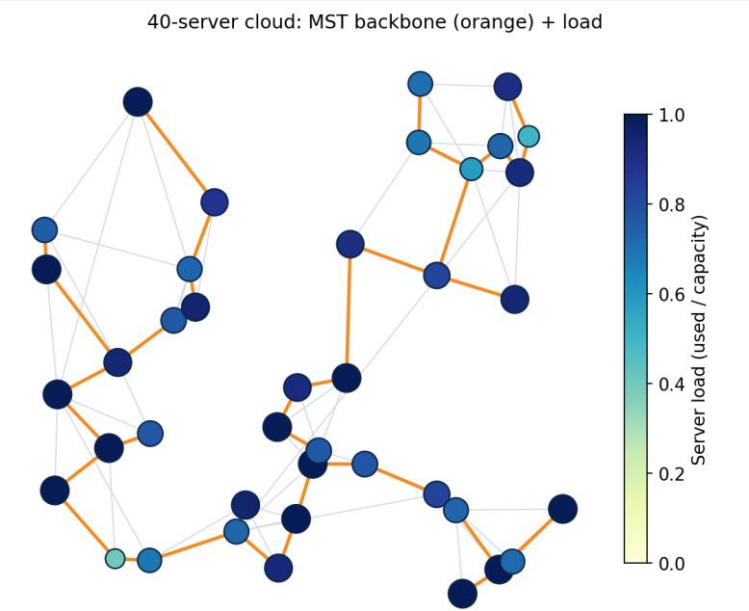
Why it is correct

- Feasible: a server accepts only a knapsack subset that fits its remaining capacity, so capacity is never exceeded (checked in code)
- Terminates: every rejection bans one server for that task, so at most V rounds
- Locally optimal: DP is exact for each server's proposals
- Not globally optimal: the problem is a generalised assignment problem (NP-hard), so we compare with an exact solver

Implementation and prototype

- **graph.py**
  Adjacency-list graph, topology and workload generators
- **mst.py**
  Kruskal with union-find, Prim with binary heap
- **shortest.py**
  Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson
- **greedy.py / dp.py**
  Fractional knapsack, job sequencing, 0-1 knapsack
- **placement.py**
  Greedy+DP, baselines, exact MILP, metrics

406 lines of source **7/7** tests pass



Demo: 120 tasks on 40 servers, 87.5% placed, 1.72 ms average latency

Experiment setup

1

Runtime vs size
25 to 200 servers, five graph algorithms timed

2

Sparse vs dense
120 servers, 191 to 3427 links, Floyd-Warshall vs Johnson

3

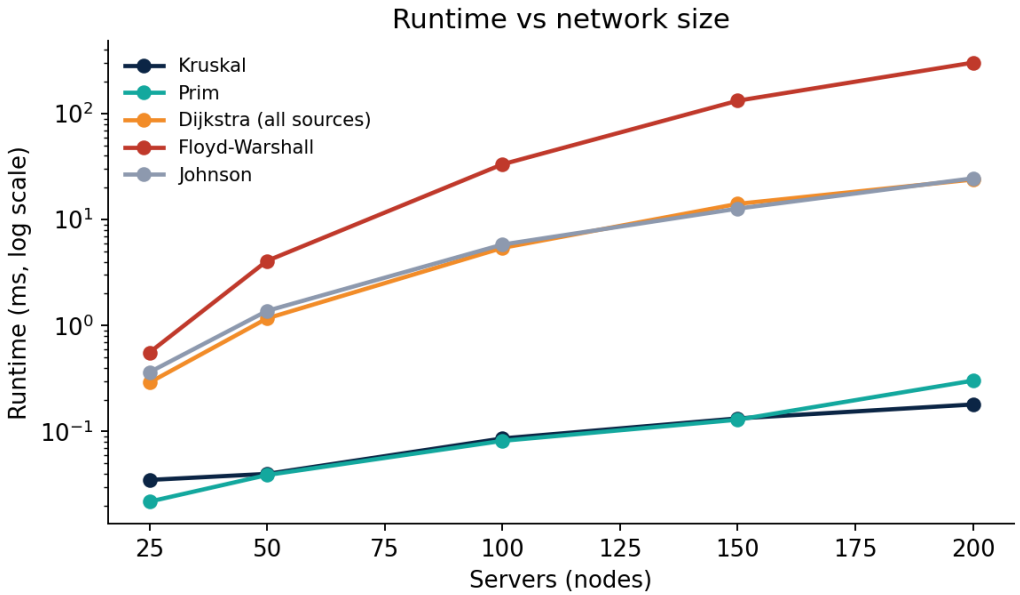
Quality vs optimum
10 servers, 16 tasks, 12 instances, exact MILP as yardstick

4

Behaviour under load
60 servers, 250 tasks, load 0.4 to 1.5, 4 seeds averaged

Metrics: total utility, % of tasks placed, average latency, capacity used, runtime. **Baselines:** nearest-first (by value) and random placement.

Results: algorithm runtime



12x

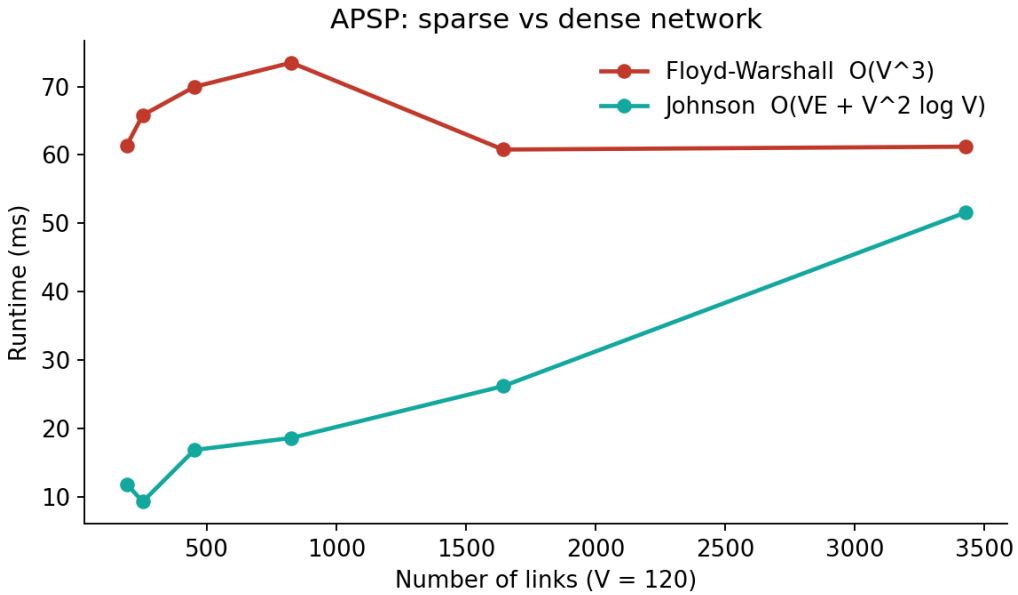
Johnson vs Floyd-Warshall at 200 servers (about 25 ms vs 303 ms)

< 0.4 ms

Kruskal and Prim at 200 servers

Single run per size on one machine, so treat small differences as noise.

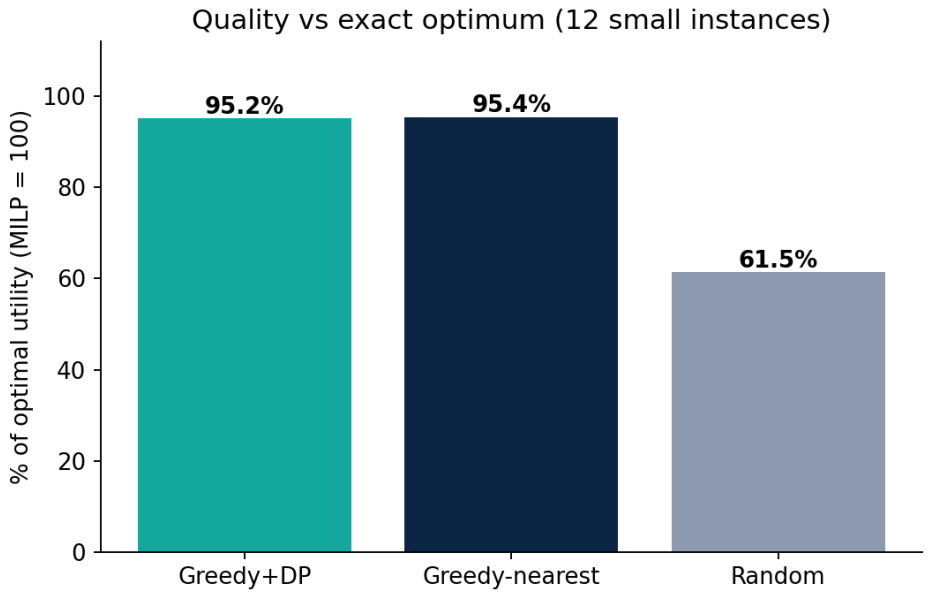
Results: sparse vs dense networks



What we saw

- 191 links: Johnson about 5x faster
- 3427 links: about 1.2x faster
- Floyd-Warshall stays roughly constant, as O(V^3) predicts
- Rule: sparse network, use Johnson

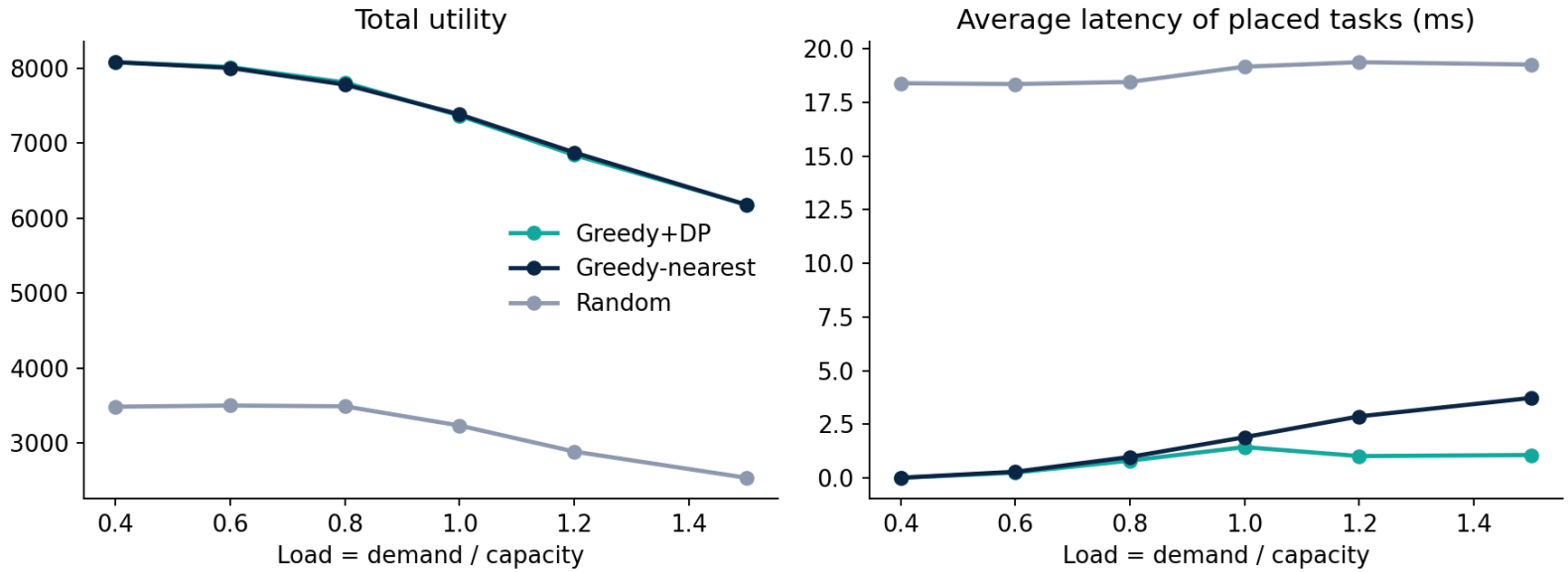
Results: solution quality



Reading the chart

- Optimum from an exact MILP solver (10 servers, 16 tasks)
- Greedy+DP: 95.2% (worst case 90.9%)
- Nearest-first: 95.4% (worst case 88.0%)
- Random: 61.5%

Results: behaviour under load



At load 1.5: Greedy+DP averages 1.07 ms vs 3.73 ms for nearest-first, placing 70.6% vs 67.9% of tasks, with the same utility (6182 vs 6179).

Policy recommendations

Servers	Links (full)	Links (MST)	Link cost cut	Latency stretch
30	109	29	83.6%	1.76x
60	228	59	82.6%	2.42x
100	371	99	81.9%	2.68x

Backbone: Use the MST for replication and control traffic; route user traffic on shortest paths.

Latency matrix: Use Johnson on sparse networks, Floyd-Warshall only when links approach V^2.

Placement: Use Greedy+DP once load passes 1.0: same utility, far lower latency.

Capacity: Utility fell from 8080 to 6182 as load rose from 0.4 to 1.5: keep headroom.

Conclusion and future work

Findings

- Pipeline of MST, shortest paths, greedy and DP solves placement end to end
- About 95% of optimal on small cases
- Best at keeping latency low under load

Limitations

- Synthetic topology and workload
- One resource type per task
- Static workload, no arrivals
- Heuristic, not provably optimal

Future work

- Multi-resource (CPU, memory) demands
- Real network traces
- Online arrivals and migration
- Replica placement

Bottom line: each textbook algorithm has a clear role, and measuring them showed where each one wins and where it does not.